

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Иркутский государственный университет»
(ФГБОУ ВО «ИГУ»)

Физический факультет
Кафедра теоретической физики
Зав. кафедрой доцент, к.ф.-м.н.,
Ловцов С.В. _____

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
«Научно-исследовательская работа»

**Обработка данных для выявления качественной
характеристики нейтринных осцилляций**

Руководитель практики: _____
Ломов В.П., к.ф.-м.н., доцент

Студент гр. 01311-ДБ
_____/Данеко Илья Игоревич

Работа защищена с оценкой _____
«___» _____ 2025г.

Нормконтролёр
_____/Фролова А.С./

Иркутск 2025

Содержание

Введение	3
1 Обработка данных численного расчёта уравнения осцилляций нейтрино в среде	4
Заключение	10
Список литературы	11
Приложение А	12

Введение

Нейтринная физика сейчас является одним из рубежей современной физики. Также благодаря своим необычным свойствам нейтрино являются важным элементом для построения моделей за пределами стандартной модели.

Нейтрино — это общее название нейтральных фундаментальных частиц с полуцелым спином, участвующих только в слабом и гравитационном взаимодействиях, и относящихся к классу лептонов. Наиболее важным открытием в физике нейтрино стало наблюдение осцилляций, которые к настоящему моменту подтверждены многочисленными экспериментами с солнечными, атмосферными, реакторными и ускорительными нейтрино.

Идея нейтринных осцилляций была предложена Бруно Понтекорво в 1957-1958 годах, она была основана по аналогии с К-мезонами [1, 2]. Уже в 1962 году, Маки, Накагавой и Сакатой была разработана теория смешивания нейтрино в вакууме [3]. В 1963 году Накагавой, Оконики, Сакатой и Тойодой была предложена связь между перемешиванием нейтрино с определенным флейвором и нейтрино с определённой массой [4]. Вольфенштейном было обращено внимание на эффект при распространении нейтрино через обычное вещество (среда электронейтральная, немагнитная, нерелятивистская) в 1978 году [5, 6]. В 1986 году Михеевым и Смирновым этот эффект был физически описан (MSW-эффект) [7, 8]. И наконец, в 2015 году в качестве доказательства осцилляций нейтрино были признаны результаты SNO (Садберийская нейтринная обсерватория) [9] и Super-Kamiokande [10].

По сравнению с вакуумными осцилляциями нейтрино, при изучении осцилляции нейтрино в веществе, все сложнее. Поэтому в таком случае, как правило, применяют численные расчёты.

Основной целью данной работы стоит поиск качественной характеристики численного решения уравнения осцилляции нейтрино в среде.

В первом разделе мы кратко рассматриваем алгоритм обработки данных численного решения уравнения осцилляций в среде полученного с помощью метода DOPRI

1. Обработка данных численного расчёта уравнения осцилляций нейтрино в среде

Поставлена задача найти качественные характеристики численного решения уравнения осцилляции нейтрино в среде

$$i \frac{d\Psi(\xi)}{d\xi} = [H_0 + v(\xi)W]\Psi(\xi), \quad (1)$$

с начальным условием

$$\Psi(\xi_0) = \begin{pmatrix} c_{13}c_{12} \\ s_{12}c_{13} \\ s_{13} \end{pmatrix}. \quad (2)$$

Из численного анализа мы заметили, что реальная и мнимая часть ψ_1 часто пересекают ось абсцисс, однако к правой границе области они выходят на асимптотическое поведение, более не пересекая ось абсцисс. Из подобных соображений было принято решение узнать как распределены нули ψ_1 . Для этого было введено понятие квазипериода. Квазипериод — это длина отрезка содержащего в себе 3 точки в которых функция (в нашем случае реальная или мнимая часть ψ_1) зануляется, причём 2 из них являются границами этого отрезка.

Мы можем считать квазипериод функцией от точки ξ_0 интервала $[0,1; 1]$ в таком смысле: находим 3 ближайших к ξ_0 значения ξ при которых функция зависящая от ξ (в нашем случае реальная или мнимая часть $\psi_1(\xi)$) зануляется. Расстояние между крайними из них и есть искомый квазипериод в точке ξ_0 .

Нас заинтересовало поведение квазипериода на всём интервале $[0,1; 1]$. Вполне логично построить график зависимости квазипериода от ξ . Для получения данных по всему интервалу мы воспользовались тем, что на правом краю интервала ψ_1 выходит на асимптотическое поведение более не пересекая ось абсцисс. Это означает, что есть самый крайний правый интервал с нулями и самый крайний квазипериод. Начиная с него и двигаясь влево мы получили необходимые данные. Для этого мы находим крайний правый квазипериод. Следующим этапом берём интервал последнего вычисленного квазипериода и смещаем его на $\frac{2}{3}$ последнего вычисленного квазипериода влево. Далее ищем все зануления функции зависящей от ξ (в нашем случае реальной и мнимой части $\psi_1(\xi)$) внутри получившегося интервала и ищем квазипериод по приведённому выше алгоритму приравняв середину получившегося интервала к ξ_0 . Повторяем этапы начиная с получения нового интервала до тех пор пока не выполняться одно из двух условий. Либо в новом интервале окажется меньше 3 точек, либо количество вычисленных квазипериодов превысит заранее заданное число (мы выбрали 100). Это делается с помощью функции "funcQPeriodStat" в приведённом в прил. (А) коде для textsfMathematica.

Нам стало интересно насколько чаще реальная и мнимая часть $\psi_{2,3}$ пересекают ось абсцисс по сравнению с реальной и мнимой частью ψ_1 соответственно. Для этого мы посчитали количество занулений реальной и мнимой частей $\psi_{2,3}$ на интервалах ранее посчитанных квазипериодов реальной и мнимой частей ψ_1 соответственно. Это делается с помощью функции

"funcCountZrsInRanges" в приведённом в прил. (А) коде для textsfMathematica.

Схожий характер графиков длин квазипериодов ψ_1 и графиков количества переходов $\psi_{2,3}$ через ноль в квазипериоде ψ_1 позволяет отнормировать данные второй и третьей компоненты по первой. Так, поделив количество нулей $\psi_{2,3}$ в каждом квазипериоде ψ_1 на длину этого квазипериода, мы получим для каждого квазипериода ψ_1 количество нулей $\psi_{2,3}$ на единицу длины, или же величину обратную среднему расстоянию между нулями. Вид полученных таким образом графиков очень похож на почти постоянный, если исключить выбросы. Но из-за этих самых выбросов масштаб оси $\frac{N_{\text{zero}}}{T}$ очень велик, поэтому чтобы убедиться в почти постоянстве величины необходимо построить графики не включающие выбросы рис. (1) (2).

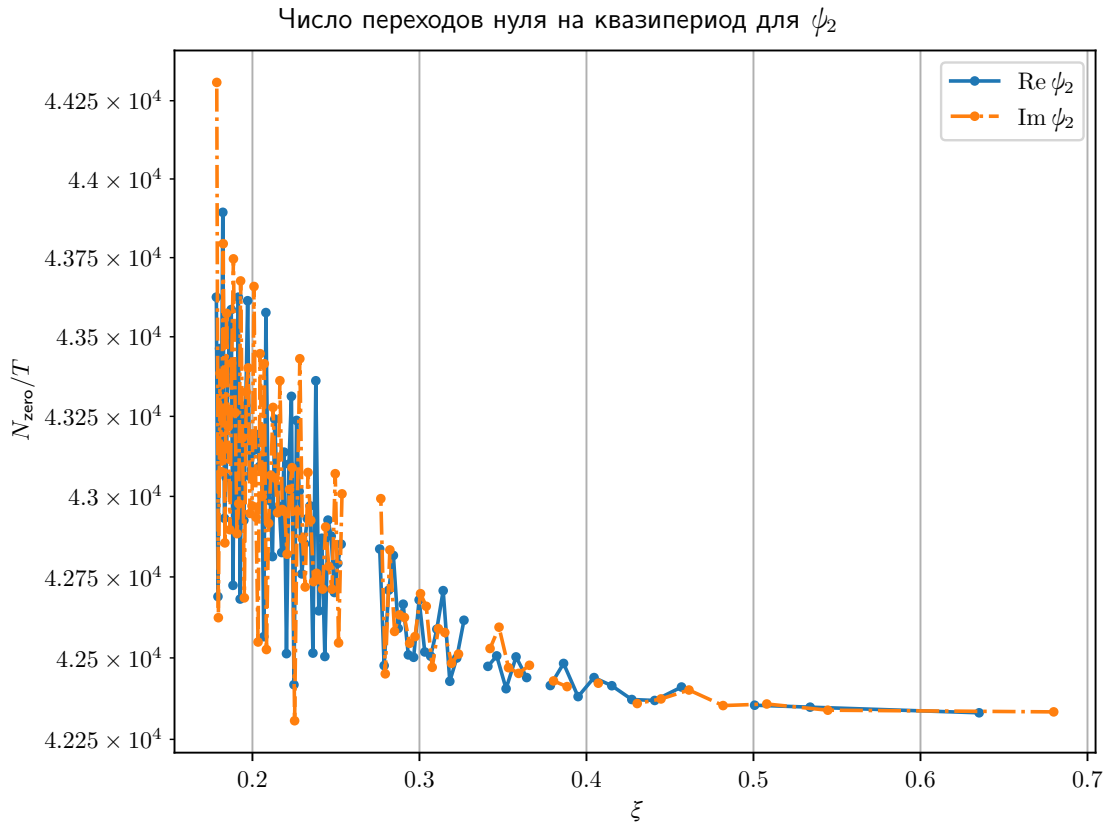


Рисунок 1 — График количество нулей $\text{Re } \psi_2$ и $\text{Im } \psi_2$ на единицу длины квазипериода ψ_1 по данным DOPRI, без выбросов

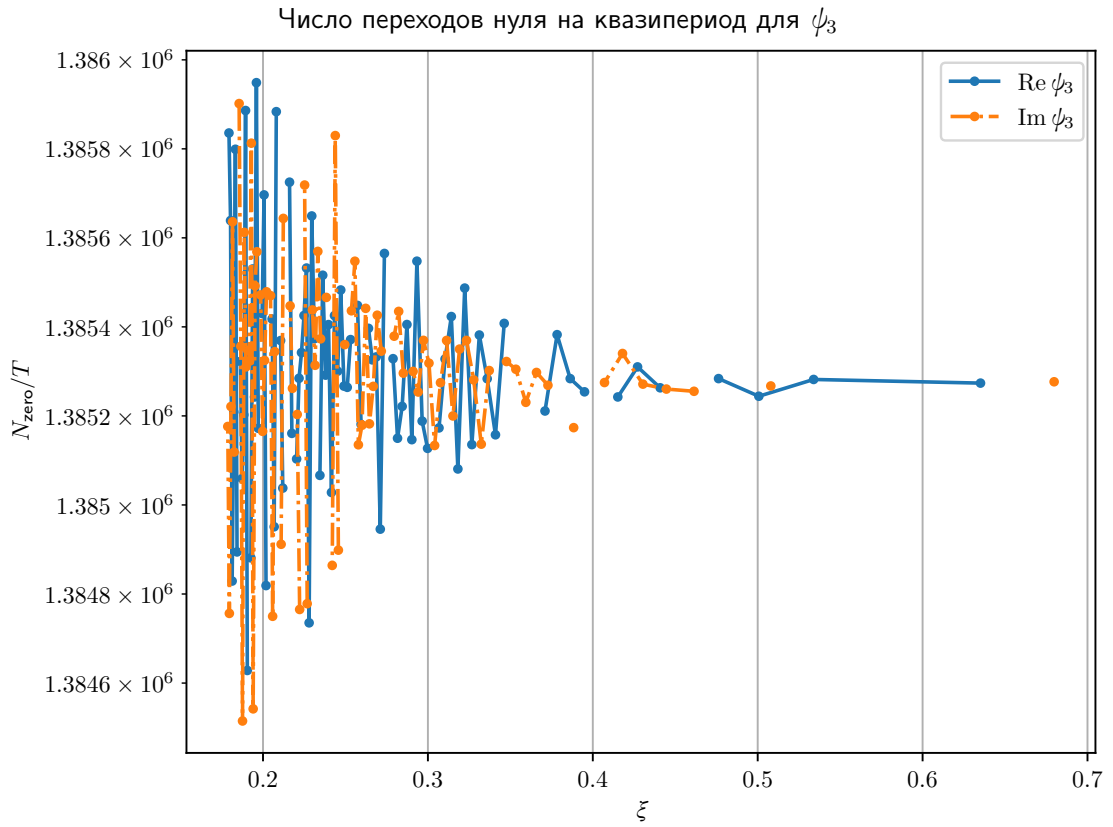


Рисунок 2 — График количество нулей $\text{Re } \psi_3$ и $\text{Im } \psi_3$ на единицу длины квазипериода ψ_1 по данным DOPRI, без выбросов

Как мы видим величины относительно постоянны и это можно признать качественным свойством. Вид графиков показывает, что это величина относительно стабильная и может быть использована для качественной характеристики численного решения. Кроме того, для реальной и мнимой частей этот параметр практически один и тот же. Аналогичное поведение мы наблюдаем и для третьей компоненты.

Графики без выбросов строились с помощью приведенного ниже кода.

```

1  #!/usr/bin/python
2
3  # import numpy as np
4  import sys
5  import matplotlib.pyplot as plt
6  from pathlib import Path
7
8  import dnk_stat as ds
9  from dnk_load import loadData2 as loadData
10
11
12  plt.rcParams["text.usetex"] = True
13  plt.rcParams["text.latex.preamble"] = (
14      r"\usepackage[conf=matplotlib]{isu-th3}"
15  )
16
17
18  def main(args):
19      """
20      'Main' function.
21      """

```

```

22
23 if len(args) < 2:
24     sys.exit("Pass at least one argument --- file name of form 'qperiod_I|Rpsi1.dat'.")
25
26 rxi, ixi, rwd, iwd, rpsi2c, ipsi2c, rpsi3c, ipsi3c = loadData(args[1])
27
28 mrk = "figs"
29 outdir = Path(mrk)
30 outdir.mkdir(parents = True, exist_ok = True)
31
32 # #####
33
34 ds.plot_pair(rxi, rwd, ixi, iwd, laby1 = r'\(T)',
35             leg1 = r'\(\Re{\psi_1})', leg2 = r'\(\Im{\psi_1})',
36             tit = r'Quasi-period of \(\psi_1)',
37             name = f"{mrk}/RIpsi1_width")
38
39 ds.plot_pair(rxi, rwd, ixi, iwd, laby1 = r'\(T)',
40             leg1 = r'\(\Re{\psi_1})', leg2 = r'\(\Im{\psi_1})',
41             tit = r'Квазипериод \(\psi_1)',
42             name = f"{mrk}/RIpsi1_width-ru")
43
44 ds.plot_l2pair(rxi, rwd, ixi, iwd, laby1 = r'\(T)',
45             leg1 = r'\(\Re{\psi_1})', leg2 = r'\(\Im{\psi_1})',
46             tit = r'Quasi-period of \(\psi_1)',
47             name = f"{mrk}/RIpsi1_logwidth")
48
49 ds.plot_l2pair(rxi, rwd, ixi, iwd, laby1 = r'\(T)',
50             leg1 = r'\(\Re{\psi_1})', leg2 = r'\(\Im{\psi_1})',
51             tit = r'Квазипериод \(\psi_1)',
52             name = f"{mrk}/RIpsi1_logwidth-ru")
53
54 # #####
55
56 ds.plot_pair(rxi, rpsi2c, ixi, ipsi2c, laby1 = r'\(N_{\text{zero}})',
57             leg1 = r'\(\Re{\psi_2})', leg2 = r'\(\Im{\psi_2})',
58             tit = r'Number of zero transitions for \(\psi_2)',
59             name = f"{mrk}/RIpsi2_trans")
60
61 ds.plot_pair(rxi, rpsi2c, ixi, ipsi2c, laby1 = r'\(N_{\text{zero}})',
62             leg1 = r'\(\Re{\psi_2})', leg2 = r'\(\Im{\psi_2})',
63             tit = r'Число переходов нуля для \(\psi_2)',
64             name = f"{mrk}/RIpsi2_trans-ru")
65
66 # #####
67
68 # ### Due to computational costs for psi3 we compute data for not whole psi1.
69 sz = len(rpsi3c)
70 rxi_r = rxi[-sz:]
71 ixi_r = ixi[-sz:]
72
73 ds.plot_pair(rxi_r, rpsi3c, ixi_r, ipsi3c, laby1 = r'\(N_{\text{zero}})',
74             leg1 = r'\(\Re{\psi_3})', leg2 = r'\(\Im{\psi_3})',
75             tit = r'Number of zero transitions for \(\psi_3)',
76             name = f"{mrk}/RIpsi3_trans")
77
78 ds.plot_pair(rxi_r, rpsi3c, ixi_r, ipsi3c, laby1 = r'\(N_{\text{zero}})',
79             leg1 = r'\(\Re{\psi_3})', leg2 = r'\(\Im{\psi_3})',
80             tit = r'Число переходов нуля для \(\psi_3)',
81             name = f"{mrk}/RIpsi3_trans-ru")
82
83 # #####
84

```

```

85 ds.plot_pair(rxi, rpsi2c/rwd, ixi, ipsi2c/iwd, laby1 = r'\(N_{\text{zero}}/T)\',
86             leg1 = r'\(\Re\{\psi_2\}\)', leg2 = r'\(\Im\{\psi_2\}\)',
87             tit = r'Number of transitions per quasi-period for \(\psi_2\)\',
88             name = f"{mrk}/RIpsi2_rel-trans")
89
90 ds.plot_pair(rxi, rpsi2c/rwd, ixi, ipsi2c/iwd, laby1 = r'\(N_{\text{zero}}/T)\',
91             leg1 = r'\(\Re\{\psi_2\}\)', leg2 = r'\(\Im\{\psi_2\}\)',
92             tit = r'Число переходов нуля на квазипериод для \(\psi_2\)\',
93             name = f"{mrk}/RIpsi2_rel-trans-ru")
94
95 ds.plot_l2pair(rxi, rpsi2c/rwd, ixi, ipsi2c/iwd, laby1 = r'\(N_{\text{zero}}/T)\',
96               leg1 = r'\(\Re\{\psi_2\}\)', leg2 = r'\(\Im\{\psi_2\}\)',
97               tit = r'Number of transitions per quasi-period for \(\psi_2\)\',
98               name = f"{mrk}/RIpsi2_logrel-trans")
99
100 ds.plot_l2pair(rxi, rpsi2c/rwd, ixi, ipsi2c/iwd, laby1 = r'\(N_{\text{zero}}/T)\',
101               leg1 = r'\(\Re\{\psi_2\}\)', leg2 = r'\(\Im\{\psi_2\}\)',
102               tit = r'Число переходов нуля на квазипериод для \(\psi_2\)\',
103               name = f"{mrk}/RIpsi2_logrel-trans-ru")
104
105 ds.plot_l2pair(rxi, rpsi2c/rwd, ixi, ipsi2c/iwd, cutoff = 1e4,
106               laby1 = r'\(N_{\text{zero}}/T)\',
107               leg1 = r'\(\Re\{\psi_2\}\)', leg2 = r'\(\Im\{\psi_2\}\)',
108               tit = r'Number of transitions per quasi-period for \(\psi_2\)\',
109               name = f"{mrk}/RIpsi2_logrel-transCUT")
110
111 ds.plot_l2pair(rxi, rpsi2c/rwd, ixi, ipsi2c/iwd, cutoff = 1e4,
112               laby1 = r'\(N_{\text{zero}}/T)\',
113               leg1 = r'\(\Re\{\psi_2\}\)', leg2 = r'\(\Im\{\psi_2\}\)',
114               tit = r'Число переходов нуля на квазипериод для \(\psi_2\)\',
115               name = f"{mrk}/RIpsi2_logrel-transCUT-ru")
116
117 # #####
118 rwd_r = rwd[-sz:]
119 iwd_r = iwd[-sz:]
120
121 ds.plot_pair(rxi_r, rpsi3c/rwd_r, ixi_r, ipsi3c/iwd_r,
122             laby1 = r'\(N_{\text{zero}})\',
123             leg1 = r'\(\Re\{\psi_3\}\)', leg2 = r'\(\Im\{\psi_3\}\)',
124             tit = r'Number of transitions per quasi-period for \(\psi_3\)\',
125             name = f"{mrk}/RIpsi3_rel-trans")
126
127 ds.plot_pair(rxi_r, rpsi3c/rwd_r, ixi_r, ipsi3c/iwd_r,
128             laby1 = r'\(N_{\text{zero}})\',
129             leg1 = r'\(\Re\{\psi_3\}\)', leg2 = r'\(\Im\{\psi_3\}\)',
130             tit = r'Число переходов нуля на квазипериод для \(\psi_3\)\',
131             name = f"{mrk}/RIpsi3_rel-trans-ru")
132
133 ds.plot_l2pair(rxi_r, rpsi3c/rwd_r, ixi_r, ipsi3c/iwd_r, laby1 = r'\(N_{\text{zero}})\',
134               leg1 = r'\(\Re\{\psi_3\}\)', leg2 = r'\(\Im\{\psi_3\}\)',
135               tit = r'Number of transitions per quasi-period for \(\psi_3\)\',
136               name = f"{mrk}/RIpsi3_logrel-trans")
137
138 ds.plot_l2pair(rxi_r, rpsi3c/rwd_r, ixi_r, ipsi3c/iwd_r, laby1 = r'\(N_{\text{zero}})\',
139               leg1 = r'\(\Re\{\psi_3\}\)', leg2 = r'\(\Im\{\psi_3\}\)',
140               tit = r'Число переходов нуля на квазипериод для \(\psi_3\)\',
141               name = f"{mrk}/RIpsi3_logrel-trans-ru")
142
143 ds.plot_l2pair(rxi_r, rpsi3c/rwd_r, ixi_r, ipsi3c/iwd_r, cutoff = 1e6,
144               laby1 = r'\(N_{\text{zero}})\',
145               leg1 = r'\(\Re\{\psi_3\}\)', leg2 = r'\(\Im\{\psi_3\}\)',
146               tit = r'Number of transitions per quasi-period for \(\psi_3\)\',
147               name = f"{mrk}/RIpsi3_logrel-transCUT")

```



```

148
149     ds.plot_l2pair(rxi_r, rpsi3c/rwd_r, ixi_r, ipsi3c/iwd_r, cutoff = 1e6,
150                   laby1 = r'\(N_{\text{zero}}\)',
151                   leg1 = r'\(\text{Re}\{\psi_3\}\)', leg2 = r'\(\text{Im}\{\psi_3\}\)',
152                   tit = r'Число переходов нуля на квазипериод для \(\psi_3\)',
153                   name = f"{mrk}/RIpsi3_logrel-transCUT-ru")
154
155
156 if __name__ == "__main__":
157     try:
158         main(sys.argv)
159     except KeyboardInterrupt:
160         sys.exit("Interrupted by user.")
161
162 # vim: fdm=indent:

```

Заключение

В работе рассматривались обработанные данные численного расчёта уравнения осцилляции нейтрино. Это позволило нам убедиться, что количество нулевых значений реальной и мнимой части $\psi_{2,3}$ на единицу длины квазипериода реальной и мнимой части ψ_1 действительно является качественной характеристикой численного решения.

Кроме этого отметим, что использование готовых решений для численного интегрирования, например, процедуры `NDSolve` программы **Mathematica** затрудняет воспроизводимость результатов, если только не задавать специально параметры для этой процедуры. К тому же, получаемый результат не представляет собой решение. Возвращаемый процедурой `NDSolve` результат — это интерполяционная формула. Но точных её параметры и другие характеристики неизвестны.

Напоследок отметим, что для уменьшения риска человеческой ошибки были разработаны сценарии для автоматизации большинства операций.

Список литературы

- [1] Pontecorvo B. Mesonium and anti-mesonium / B. Pontecorvo // Sov. Phys. JETC. — 1957. — Т. 6. — С. 429.
- [2] Pontecorvo B. Inverse beta processes and nonconservation of lepton charge / B. Pontecorvo // Sov. Phys. JETC. — 1958. — Т. 7. — С. 172–173.
- [3] Maki Z. Remarks on the unified model of elementary particles / Z. Maki, M. Nakagawa, S. Sakata // High-energy physics. Proceedings, 11th International Conference, ICHEP'62, Geneva, Switzerland, Jul 4-11, 1962. — С. 663–666.
- [4] Possible existence of a neutrino with mass and partial conservation of muon charge / M. Nakagawa [et al.] // Prog. Theor. Phys. — 1963. — Т. 30. — С. 727–729.
- [5] Wolfenstein L. Neutrino Oscillations in Matter / L.Wolfenstein // Phys. Rev. — 1978. — Т. D17. — С. 2369–2374.
- [6] Wolfenstein L. Effects of Matter on Neutrino Oscillations / L.Wolfenstein // AIP Conf. Proc. — 1978. — Т. 52. — С. 108–112.
- [7] Mikheev S. P. Neutrino oscillations in matter / S. P. Mikheev, A. Y. Smirnov // 12th International Conference on Neutrino Physics and Astrophysics (Neutrino 86) Sendai, Japan, June 3-8, 1986. — С. 177–193.
- [8] Mikheev S. P. Resonance Oscillations of Neutrinos in Matter / S. P. Mikheev, A. Y. Smirnov // Sov. Phys. Usp. — 1987. — Т. 30. — С. 759–790.
- [9] Measurement of the rate of $\nu_e + d \rightarrow p + p + e^-$ interactions produced by 8B solar neutrinos at the Sudbury Neutrino Observatory / Q. R. Ahmad [et al.] // Phys. Rev. Lett. — 2001. — Т. 87. — С. 071301. —
- [10] Evidence for oscillation of atmospheric neutrinos / Y. Fukuda [et al.] // Phys. Rev. Lett. — 1998. — Т. 81. — С. 1562–1567.
- [11] Casas F. Efficient numerical integration of neutrino oscillations in matter / F. Casas, J.C. D'Olivo, J.A.Oteo // Phys. Rev. D. — 2016. — Т. 94, no. 11. — С. 113008.

Код в Wolfram Mathematica для решения уравнения осцилляции нейтрино в среде

Для численного решения уравнения (1) использовалась система компьютерной алгебры Wolfram Mathematica.

Ниже представлен код сценария, использованного для получения данных по квазипериодам $\text{Re } \psi_1(\xi)$, $\text{Im } \psi_1(\xi)$ в зависимости от ξ , а также данных по количеству переходов через ось абсцисс $\text{Re } \psi_{2,3}$ и $\text{Im } \psi_{2,3}$ на интервале каждого из ранее найденных квазипериодов $\text{Re } \psi_1$ и $\text{Im } \psi_1$ соответственно.

Его также можно найти в репозитории <https://github.com/Fest-Fut/Kursovoy>.

```

1  ### Global Parameters
2
3  We are researching the peculiarities of numeric solution of the neutrino oscillation in matter equation obtained by
  ↳ Runge--Kutta family of methods.
4
5  The equation relies on the Hermitian matrix  $(H=H_0 + v(\xi)W)$ , where  $(H_0)$  is Hermitian matrix of oscillation in
  ↳ vacuum, the  $(v(\xi))$  is density function (effective density profile) and  $(W)$  is composed from mixing angles.
6
7  Due to its nature, the oscillation depends on neutrino energy, which is incorporated into  $(H)$  matrix, while the
  ↳ neutrino masses are part of  $(H_0)$  matrix. Below we set these parameters and though we explicitly set energy to
  ↳ only one value, it is possible, after testing, to adapt the code to any energy (making it function-like).
8
9  ```mathematica
10 a = 4.35196 10^6;
11 b = 0.030554;
12 ```
13
14 #### Sun model
15
16 As for the density function, we use known result for the Sun (aka, the Sun model). The density is exponential function
  ↳ with given factor and known magnitude factor.
17
18 ```mathematica
19  $\rho[\text{Gamma}] = 6.5956 10^4$ ;
20 n = 10.54;
21 u[ $\rho[\text{Xi}]$ ] :=  $\rho[\text{Gamma}] \text{Exp}[ -n \rho[\text{Xi}] ]$ 
22 ```
23
24 #### The mixing angles
25
26 The  $(W)$  matrix, as was mentioned above, is constructed from mixing angles. The formula is given below. We use values
  ↳ for the angles from the work [Casas et al, 2016].
27
28 ```mathematica
29 s12 = Sqrt[0.308];
30 s23 = Sqrt[0.437];
31 s13 = Sqrt[0.0234];
32 c12 = Sqrt[1 - s12^2];
33 c23 = Sqrt[1 - s23^2];
34 c13 = Sqrt[1 - s13^2];
35 W = {{c13^2 c12^2, c12 s12 c13^2, c12 c13 s13}, {c12 s12 c13^2, s12^2 c13^2, s12 c13 s13}, {c12 s13 c13, s12 c13 s13,
  ↳ s13^2}};
36 W // MatrixForm;
37

```

```

38  ***
39
40  #### Dorman-Prince method
41
42  According to work [Casas et al], the more “correct” result would be obtained if one uses the Dorman-Prince method of
  ↳ Runge--Kutta family. Mathematica allows to explicitly set coefficients for Runge--Kutta method used internally.
43
44  ```mathematica
45  DOPRICoeffs[5, p_] := N[{{1/5}, {3/40, 9/40}, {44/45, -56/15, 32/9}, {19372/6561, -25360/2187, 64448/6561, -212/729},
  ↳ {9017/3168, -355/33, 46732/5247, 49/176, -5103/18656}, {35/384, 0, 500/1113, 125/192, -2187/6784, 11/84}}, {35/384,
  ↳ 0, 500/1113, 125/192, -2187/6784, 11/84, 0}, {1/5, 3/10, 4/5, 8/9, 1, 1}, {71/57600, 0, -71/16695, 71/1920,
  ↳ -17253/339200, 22/525, -1/40}}, p];
46  ***
47
48  ```mathematica
49  Ψ[Psi][Xi_] := {Ψ[Psi]1[Xi], Ψ[Psi]2[Xi], Ψ[Psi]3[Xi]}
50  ***
51
52  ```mathematica
53  (* The idea is following: the getPsi function should calculate solution for given energy and returns the function
  ↳ handler, which could be used to get actual numbers. All used variables must be local, but it may be incorrect way to
  ↳ use `Module`, because all it's variables are local, so it for uf... dunno. May be we should use `Block`, but before
  ↳ that we must compare `Block` vs `Module`. *)
54  ***
55
56  ```mathematica
57  (* This function must all the work that the previous version of NB did (current one is p5, see p4). *)
58  ***
59
60  ```mathematica
61  getPsi[energy_] := Module[{a = 4.35196 10^-6, b = 0.030554,
62    H0, W, u, Ψ[Psi], Ψ[Psi]1, Ψ[Psi]2, Ψ[Psi]3, sol, uf, Ψ[Xi]0 = 0.1, Ψ[Xi]1 = 1.0, DOPRICoeffs,
63    s12 = Sqrt[0.308], s23 = Sqrt[0.437], s13 = Sqrt[0.0234],
64    c12 = 0, c13 = 0, c23 = 0, Ψ[Gamma] = 6.5956 10^-4, n = 10.54,
65  },
66    c12 = Sqrt[1 - s12^2];
67    c23 = Sqrt[1 - s23^2];
68    c13 = Sqrt[1 - s13^2];
69    H0 = a/energy {{0, 0, 0}, {0, b, 0}, {0, 0, 1}};
70    W = {{c13^2 c12^2, c12 s12 c13^2, c12 c13 s13}, {c12 s12 c13^2, s12^2 c13^2, s12 c13 s13}, {c12 s13 c13, s12 c13
  ↳ s13, s13^2}};
71    H0;
72    u[Ψ[Xi]] := Ψ[Gamma] Ψ[ExponentialE]^(-n Ψ[Xi]);
73    Ψ[Psi][Xi_] := {Ψ[Psi]1[Xi], Ψ[Psi]2[Xi], Ψ[Psi]3[Xi]};
74    DOPRICoeffs[5, p_] := N[{{1/5}, {3/40, 9/40}, {44/45, -56/15, 32/9}, {19372/6561, -25360/2187, 64448/6561,
  ↳ -212/729}, {9017/3168, -355/33, 46732/5247, 49/176, -5103/18656}, {35/384, 0, 500/1113, 125/192, -2187/6784,
  ↳ 11/84}}, {35/384, 0, 500/1113, 125/192, -2187/6784, 11/84, 0}, {1/5, 3/10, 4/5, 8/9, 1, 1}, {71/57600, 0,
  ↳ -71/16695, 71/1920, -17253/339200, 22/525, -1/40}}, p];
75    sol = NDSolve[{Ψ[ImaginaryI]*D[Ψ[Psi][Xi], Ψ[Xi]] == (H0 + u[Ψ[Xi]]*W) Ψ[Psi][Xi], Ψ[Psi][Xi]0 == {c12
  ↳ c13, s12 c13, s13}}, {Ψ[Psi]1, Ψ[Psi]2, Ψ[Psi]3}, {Ψ[Xi], Ψ[Xi]0, Ψ[Xi]1, Method -> {"ExplicitRungeKutta",
  ↳ "Coefficients" -> DOPRICoeffs, "DifferenceOrder" -> 5, "StiffnessTest" -> False}, StartingStepSize -> 0.25,
  ↳ MaxSteps -> Infinity, MaxStepFraction -> 1/12, WorkingPrecision -> MachinePrecision, AccuracyGoal -> Infinity];
76    uf[x_] := Evaluate[{Ψ[Psi]1[Xi], Ψ[Psi]2[Xi], Ψ[Psi]3[Xi]} /. sol] /. {Ψ[Xi] -> x} /. {Ψ[Xi_] -> Ψ[Xi];
77    uf
78  ];
79  (* TODO: check that function getPsi is working. Assumed interface:
80  uf=getPsi[1];
81  Then, the below (with uf) must work as in `p4` part.
82  *)
83  ***
84
85  #### Numeric solution
86

```

```

87 One of flaws of the [Casas et al] work is that they don't describe how they get the numeric solution from Mathematica.
88 ↪ For example, if one sees below, it would see that the call to `NDSolve` is quite complex. First, we use own
89 ↪ coefficients, then we set other parameters to fine tune the Mathematica `NDSolve` to obtain numeric result.
90
91 ```mathematica
92
93 ```mathematica
94 sol = NDSolve[{I*Dt[Psi][Xi], Psi == (H0 + u[Xi]*W) . Psi, Psi[Xi0] == {c12 c13,
95 ↪ s12 c13, s13}}, {Psi1, Psi2, Psi3}, {Xi, Xi0, Xi1}, Method -> {"ExplicitRungeKutta",
96 ↪ "Coefficients" -> DOPRICoeffs, "DifferenceOrder" -> 5, "StiffnessTest" -> False}, StartingStepSize -> 0.25, MaxSteps
97 ↪ -> Infinity, MaxStepFraction -> 1/12, WorkingPrecision -> MachinePrecision, AccuracyGoal -> Infinity];
98
99 #### The solution form
100
101 Besides opaque way how Mathematica calculates the solution, the `NDSolve` returns not a solution itself, but rather
102 ↪ interpolation function using Pade approximation (?). The details very cloudy, so we have only hunch of what is going
103 ↪ on.
104
105 Still, to get numbers from solution we have to construct the `uf` handler.
106
107 ```mathematica
108 uf[x_] := Evaluate[{Psi1[Xi], Psi2[Xi], Psi3[Xi]} /. sol] /. {Xi -> x} /. {Xi_ -> Xi};
109
110 #### The "correctness" check route
111
112 The below code is used to check the solution "correctness". Ideally, the equation is the Schrödinger type with Hermitian
113 ↪ matrix. As such, the probability (or the norm of the function) must be conserved. If initially the norm is very
114 ↪ close to 1, then the solution at any point (\\xi=1\\) including, must have the same norm.
115
116 Though, if there is an absorption then the matrix \\(H\\) is NOT Hermitian anymore and this feature is not hold.
117
118 ```mathematica
119 quf[v_] := Abs[v[[1]]]^2 + Abs[v[[2]]]^2 + Abs[v[[3]]]^2;
120
121
122 ```mathematica
123 absuf[v_] := {Abs[v[[1]]]^2, Abs[v[[2]]]^2, Abs[v[[3]]]^2};
124
125
126 ```mathematica
127 reimuf[v_] := {Re[v[[1]]], Im[v[[1]]], Re[v[[2]]], Im[v[[2]]], Re[v[[3]]], Im[v[[3]]]};
128
129
130 ```mathematica
131 surProb[v_] := c12^2*c13^2*Abs[v[[1]]]^2 + s12^2*c13^2*Abs[v[[2]]]^2 + s13^2*Abs[v[[3]]]^2;
132
133
134 ```mathematica
135 arguf[v_] := {Arg[v[[1]]], Arg[v[[2]]], Arg[v[[3]]]};
136
137
138 ```mathematica
139 st = 10^(-5);
140
141
142 ```mathematica
143 xi0 = Xi0 + 0.01; xi1 = Xi1 - 0.1;
144

```

```

141  ```mathematica
142  dat1 = Table[Flatten[{x, surProb[uf[x]], reimuf[uf[x]], arguf[uf[x]], absuf[uf[x]], quf[uf[x]] - 1}], {x, xi0, xi1,
    ↳ st}];
143  ```
144
145  ##### The data directory
146
147  It is inconvenient to use full path when exporting (saving) the data. Instead, one may use the path relative to the
    ↳ notebook itself, as we do below. It is big help that `NotebookDir[]` returns path with path separator at the end.
148
149  NB: the `<>` is the Mathematica way to join (merge) the string.
150
151  ```mathematica
152  (* SetDirectory[$UserDocumentsDirectory] ...
153  NotebookDirectory[] ...
154  $HomeDirectory ...
155  $UserBaseDirectory ...
156  $Username
157  In[.] $UserBaseDirectory
158  Out[.] -> dir *)
159  ```
160
161  ```mathematica
162  nbDir = NotebookDirectory[];
163  ```
164
165  ```mathematica
166  mkEngFmt[n_] := ToString[EngineeringForm[1. n, NumberFormat -> (Row[{#1, "e", #3}]] &)];
167  ```
168
169  ```mathematica
170  mkScnFmt[n_] := ToString[ScientificForm[1. n, NumberFormat -> (Row[{#1, "e", #3}]] &)];
171  ```
172
173  ```mathematica
174  datfn = FileNameJoin[nbDir, "SUN_DOPRI_" <> ToString[Xi][0] <> "_" <> ToString[Xi][1] <> "_" <> mkScnFmt[st] <>
    ↳ ".dat"];
175  ```
176
177  ```mathematica
178  Export[datfn, dat1, "Table"];
179  ```
180
181  ##### The convenience
182
183  For convenience we explicitly “extract” parts of solution: real and imaginary parts, and all three components.
184
185  ```mathematica
186  eRpsi1[x_] := Re[uf[x][[1]]];
187  eIpsi1[x_] := Im[uf[x][[1]]];
188  eRpsi2[x_] := Re[uf[x][[2]]];
189  eIpsi2[x_] := Im[uf[x][[2]]];
190  eRpsi3[x_] := Re[uf[x][[3]]];
191  eIpsi3[x_] := Im[uf[x][[3]]];
192  ```
193
194  ### The utility functions
195
196  These functions are used to get some data from obtained solution.
197
198  ##### The `funcNullRanges` function
199
200  Takes function handler, two points and integer.

```

```

201
202 The purpose is roughly search for the ranges where the function has a zero. The two points set the wide range within we
    ↳ looking for the zeroes. The integer parameter is used to split the wide range into pieces. The function actually
    ↳ doesn't calculate exact position of the zeroes. Instead, it checks the function signs on the subinterval edges. If
    ↳ the sign is different, then we call such interval a zero range.
203
204 The routine returns array of zero ranges.
205
206 We presume that the function has zero of first order (only!) and many of them.
207
208 ```mathematica
209 funcNullRanges[fn_, x1_, x2_, n_] := Module[{t = {}, xx = {}, mm = {}, dx = (x2 - x1)/n, i = 1, nL = 0},
210     xx = Table[k, {k, x1, x2, dx}];
211     mm = Table[fn[k], {k, xx}];
212     nL = Length[mm];
213     While[i < nL,
214         If[mm[[i]]*mm[[i + 1]] < 0, AppendTo[t, {xx[[i]], xx[[i + 1]]}]];
215         i++;];
216     t
217 ];
218 ```
219
220 ##### The `funcNullRangesStab` function
221
222 Takes the same argument as `funcNullRanges`.
223
224 The previous function is a bit too rough, so this one tries to "stabilize" the number of zero ranges.
225
226 The idea is simple: if the wide range splitting on different number of subparts gives the save number of zero ranges,
    ↳ then we found the "exact" number of "zeroes" on the wide interval. For that we iteratively call the `funcNullRanges`
    ↳ function increasing number of splits and checking the returned array size.
227
228 The routine returns array of zero ranges (as function `funcNullRanges` do).
229
230 ```mathematica
231 funcNullRangesStab[fn_, x1_, x2_, n_] := Module[{n1 = -1, n2 = -1, nx = n, i = 0, imx = 6, tt = {}},
232     While[i < imx,
233         nx = n (4 + i)^i;
234         tt = funcNullRanges[fn, x1, x2, nx];
235         n1 = n2;
236         n2 = Length[tt];
237         If[n1 == n2, Break[]];
238         i++;];
239     tt
240 ];
241 ```
242
243 ##### The `funcTochnNull` function
244
245 Takes four arguments: function handler, two points and precision.
246
247 It returns zero range which length is less than given precision.
248
249 ```mathematica
250 funcTochnNull[fn_, x1_, x2_, eps_] := Module[{t1 = x1, t2 = x2, tt = {}},
251     While[t2 - t1 > eps,
252         If[fn[t1] fn[(t1 + t2)/2] < 0, t2 = (t1 + t2)/2, t1 = (t1 + t2)/2];
253     ];
254     List[t1, t2]
255 ];
256 ```
257
258 ##### The `funcQPRangeEnvelop` function

```



```

259
260 Takes four arguments: function handler and three points.
261
262 The first point is a point in vicinity of which we try to find zero range. The rest two give the wide range bounds,
↪ within which the assumed zero range should be located.
263
264 Idea for the function is the following. We want to find the nearest to given point the function zero. We only know, that
↪ is must be somewhere in the given range. It is possible that there is more than one such zero. We need only the
↪ nearest. So we get all possible zeroes (zero ranges), subtract the given point and search for the smallest possible
↪ distance. There is a possibility that the given point resides "exactly" between two zeroes. In such case we take the
↪ left one.
265
266 ```mathematica
267 funcQPRangeEnvelop[fn_, x0_, x1_, x2_] := Module[{i0 = 0, tt = {}, x1 = {}, xc = {}, xr = {}, eps = 10^-8},
268   tt = funcNullRangesStab[fn, x1, x2, 10];
269   i0 = tt[;;, 1]] - x0 // Abs // PositionSmallest // First;
270   If[{i0 > Length[tt] - 1} || {i0 == 1},
271     (*Print[[WARNING] `funcQPRangeEnvelop`: the smallest is on an edge!];
272     Print[[DEBUG] i0 = , i0, ; x0 = , x0, ; (, x1,;,x2,); ; tt = , tt];*)
273     Return[List[-1, -1]];
274     (*Print[[DEBUG:funcQPRangeEnvelop] i0 = , i0, ; tt = , tt, ; tt[:,1] - x0 = , tt[;;,1]-x0];*)
275     x1 = tt[[i0 - 1]] // Flatten;
276     xc = tt[[i0]] // Flatten;
277     xr = tt[[i0 + 1]] // Flatten;
278     x1 = funcTochnNull[fn, x1[[1]], x1[[2]], eps];
279     xr = funcTochnNull[fn, xr[[1]], xr[[2]], eps];
280     List[x1[[1]], xr[[2]]]
281   ]
282   ```
283
284 ##### The `funcLastQPeriod` function
285
286 Takes two arguments: function handler and integer.
287
288 This is very specific function. The real and imaginary parts of the solution "oscillates" very often, but the first
↪ component (which is true for the Sun model) at the right edge of the interval ( $\xi=1$ ) stops oscillating and
↪ "asymptotically" tends to a value (zero).
289
290 This function finds the last three zeroes, such that after the last one there no more zeroes.
291
292 Returns array of zero ranges.
293
294 Designed only for real and imaginary parts of first component.
295
296 ```mathematica
297 xi0 = 0.5; xi1 = 0.99;
298 ```
299
300 ```mathematica
301 funcLastQPeriod[fn_, n_] := Module[{x0 = xi0, x2 = xi1, i = 1, tt = {}, y1 = 0, y2 = 0, y3 = 0, eps = 10^-8},
302   tt = funcNullRangesStab[fn, x0, x2, n];
303   k = Length[tt];
304   y1 = funcTochnNull[fn, tt[[k - 2, 1]], tt[[k - 2, 2]], eps];
305   y2 = funcTochnNull[fn, tt[[k - 1, 1]], tt[[k - 1, 2]], eps];
306   y3 = funcTochnNull[fn, tt[[k, 1]], tt[[k, 2]], eps];
307   List[y1, y2, y3]
308 ]
309 ```
310
311 ##### The `funcQPeriodStat` function
312
313 Takes three arguments: function handler and two integers. The first integer is a number of "rough tries", the second one
↪ is the number of random sample points.

```

```

314
315 Despite the name it works only with real and imaginary parts of first component (though technically, could work with
↪ second component too) and returns array of two element array. The first element is the random point within
↪ predefined interval, the second element is the width of zero range containing the random point.
316
317 The function handler and number of "rough tries" are passed to `funcLastQPeriod` function.
318
319 The random points are generated by `getSRandoms` function. The interval is predefined and doesn't changed:
↪ \((0,11;0,99)\).
320
321 As we are interested in how the "quasiperiod" (the width of zero range) changes while goes from left to right by
↪ \(\xi\), we collect the data in an array.
322
323 Later, it could be used by other routines to obtain data for second and third components.
324
325 ```mathematica
326 xi0 = 0.11
327 ```
328
329 ```mathematica
330 funcQPeriodStat[fn_, nSeed_, nSteps_] := Module[{i = 1, tabQPeriod = {}, endt = {}, x00 = 0, x01 = 0, x02 = 0, x0 = xi0,
↪ gran1 = {}, dx = 0, t = 0, pf = {1, 3/2, 7/4, 2}, k = 1, fl = 1},
331   endt = funcLastQPeriod[fn, nSeed];
332   dx = endt[[3, 1]] - endt[[1, 1]];
333   AppendTo[tabQPeriod, {endt[[1, 1]], endt[[3, 1]]}];
334   fl = Length[pf];
335   While[i <= nSteps,
336     t = tabQPeriod[[i, 1]];
337     x02 = t + dx/4;
338     Label[rechoose];
339     x01 = t - pf[[k]] dx;
340     x00 = (x01 + x02)/2;
341     If[x01 < x0, Print["WARNING] reached the lower bound (", x0, ") on step ", i, ""]; tabQPeriod; Break[]];
342     (*Print[[DEBUG] funcQPeriodStat] i = , i, ; (, x01, ; ,x02,); x00 = , x00, ; t = , t, ; dx = , dx];*)
343     gran1 = funcQPRangeEnvelop[fn, x00, x01, x02];
344     (*Print[[DEBUG] gran1 = , gran1, ; x00 = , x00, ; (, x01, ; ,x02,); k = , k];*)
345     If[k >= fl, Print["WARNING] rough way to get Null Enveloped Ranges failed: get ", i, " ranges."]; Goto[end]];
346     If[gran1[[1]] == -1, k = k + 1; Goto[rechoose]];
347     dx = Abs[gran1[[2]] - gran1[[1]]];
348     AppendTo[tabQPeriod, gran1];
349     i++; k = 1;];
350   Label[end];
351   tabQPeriod
352 ]
353 ```
354
355 ```mathematica
356 nSamp = 100;
357 ```
358
359 ```mathematica
360 qpRR1 = funcQPeriodStat[eRpsi1, 10, nSamp];
361 ```
362
363 ```mathematica
364 qpRI1 = funcQPeriodStat[eIpsi1, 10, nSamp];
365 ```
366
367 ```mathematica
368 datfR = nbDir <> "qperiod_Rpsi1_" <> mkScnFmt[nSamp] <> ".dat";
369 datfI = nbDir <> "qperiod_Ipsi1_" <> mkScnFmt[nSamp] <> ".dat";
370 ```
371

```

```

372 ```mathematica
373 Export[datfR, qpRR1, "Table"];
374 Export[datfI, qpRI1, "Table"];
375 ```
376
377 ```mathematica
378 tP2R = funcCountZrsInRanges[eRpsi2, qpRR1, 10];
379 ```
380
381 ```mathematica
382 tP2I = funcCountZrsInRanges[eIpsi2, qpRI1, 10];
383 ```
384
385 ```mathematica
386 tP3R = funcCountZrsInRanges[eRpsi3, qpRR1, 10];
387 ```
388
389 ```mathematica
390 tP3I = funcCountZrsInRanges[eIpsi3, qpRI1, 10];
391 ```
392
393 ```mathematica
394 dat2R = nbDir <> "zrsCountsRpsi2_" <> mkScnFmt[nSamp] <> ".dat";
395 dat2I = nbDir <> "zrsCountsIpsi2_" <> mkScnFmt[nSamp] <> ".dat";
396 dat3R = nbDir <> "zrsCountsRpsi3_" <> mkScnFmt[nSamp] <> ".dat";
397 dat3I = nbDir <> "zrsCountsIpsi3_" <> mkScnFmt[nSamp] <> ".dat";
398 ```
399
400 ```mathematica
401 Export[dat2R, tP2R, "Table"];
402 Export[dat2I, tP2I, "Table"];
403 Export[dat3R, tP3R, "Table"];
404 Export[dat3I, tP3I, "Table"];
405 ```
406
407 ```mathematica
408 nSamp = 1000;
409 ```
410
411 ```mathematica
412 datfR = nbDir <> "qperiod_Rpsi1_" <> mkScnFmt[nSamp] <> ".dat";
413 datfI = nbDir <> "qperiod_Ipsi1_" <> mkScnFmt[nSamp] <> ".dat";
414 ```
415
416 ```mathematica
417 qpTR1 = funcQPeriodStat[eRpsi1, 10, nSamp];
418 ```
419
420 ![0hxt01c3hl6e8](img/0hxt01c3hl6e8.png)
421
422 ```mathematica
423 qpTI1 = funcQPeriodStat[eIpsi1, 10, nSamp];
424 ```
425
426 ![08rzkm9ftbu09](img/08rzkm9ftbu09.png)
427
428 ```mathematica
429 Export[datfR, qpTR1, "Table"];
430 Export[datfI, qpTI1, "Table"];
431 ```
432
433 ```mathematica
434 funcCountZrsInRanges[fn_, tab_, n_] := Module[{i = 1, tabZrs = {}, l = 0, tt = {}, nm = 0},

```

```

435     l = Length[tab];
436     While[i <= l,
437         tt = funcNullRangesStab[fn, tab[[i, 1]], tab[[i, 2]], n];
438         nm = Length[tt];
439         AppendTo[tabZrs, {tab[[i, 1]], tab[[i, 2]], nm}];
440         i++;
441     ];
442     tabZrs
443 ]
444 ```
445
446 ```mathematica
447 qp2R = funcCountZrsInRanges[eRpsi2, qpTR1, 10];
448 ```
449
450 ```mathematica
451 qp2I = funcCountZrsInRanges[eIpsi2, qpTI1, 10];
452 ```
453
454 ```mathematica
455 qp3R = funcCountZrsInRanges[eRpsi3, qpTR1, 10];
456 ```
457
458 ```mathematica
459 qp3I = funcCountZrsInRanges[eIpsi3, qpTI1, 10];
460 ```
461
462 ```mathematica
463 dat2R = nbDir <> "zrsCountsRpsi2_" <> mkScnFmt[nSamp] <> ".dat";
464 dat2I = nbDir <> "zrsCountsIpsi2_" <> mkScnFmt[nSamp] <> ".dat";
465 dat3R = nbDir <> "zrsCountsRpsi3_" <> mkScnFmt[nSamp] <> ".dat";
466 dat3I = nbDir <> "zrsCountsIpsi3_" <> mkScnFmt[nSamp] <> ".dat";
467 ```
468
469 ```mathematica
470 Export[dat2R, qp2R, "Table"];
471 Export[dat2I, qp2I, "Table"];
472 Export[dat3R, qp3R, "Table"];
473 Export[dat3I, qp3I, "Table"];
474 ```

```